

Kết quả thử nghiệm và Phân tích nguyên nhân Tái hiện bài báo DRL-based Adaptive Handover Protocols

Cập nhật: 2026-07-02

Paper: *A Deep Reinforcement Learning-Based Approach for Adaptive Handover Protocols*

Authors: Johannes Voigt, Peter J. Gu, Peter M. Rost — KIT / TUM

Venue: SCC 2025

Repo gốc: <https://github.com/kitt-ai/HandoverOptimDRL>

Môi trường thực nghiệm: RTX 3060 Ti (8GB) · CUDA 12.8 · PyTorch 2.7.0+cu126 · Python 3.12 · uv

Phạm vi tài liệu này: Chỉ trình bày kết quả thử nghiệm và phân tích nguyên nhân (không bao gồm phần giới thiệu lý thuyết nền, formulation bài toán, hay kiến trúc PPO — xem COMPREHENSIVE_REPORT.pdf cho bản đầy đủ).

Cập nhật: 2026-07-02

Contents

1	Kết quả baseline (tái hiện thành công)	2
1.1	3GPP Baseline	2
1.2	PPO gốc (pre-trained model của paper)	2
1.3	So sánh với paper	2
2	Vấn đề cốt lõi: Credit Assignment Problem	2
2.1	Tại sao policy bị stuck ở uniform distribution?	3
2.2	Ước tính tín hiệu gradient trong 5M steps	3
2.3	Tại sao pre-trained model của paper lại hoạt động?	4
2.4	Thách thức 9-timestep delay	4
3	Thử nghiệm V1 — Training từ scratch	4
3.1	Thử nghiệm 1A: t _{ho_prep} =3 trong Phase 1	4
3.2	Thử nghiệm 1B: 2-phase đúng paper (t _{ho_prep} =5 xuyên suốt)	5
4	Thử nghiệm V2 — 4 phương án giải quyết	5
4.1	Phương án Curriculum t _{ho_prep} = 1 → 3 → 5	5
4.2	Phương án Reward Shaping	6
4.3	Phương án Imitation Learning (BC + PPO)	6
4.4	Phương án Multiple Seeds	7
4.5	Tổng hợp V2	7
5	Thử nghiệm V3 — BC + Gradual Curriculum	8

6	Thử nghiệm V4 — Biến thể BC+Curriculum	8
6.1	bc_paper — BC + paper params trực tiếp	9
6.2	bc_highent — BC + Curriculum + ent_coef=0.1	9
6.3	bc_noterm — BC + Curriculum + không terminate	9
6.4	bc_2m — BC + Curriculum + 2M steps/phase	10
6.5	Tổng hợp V4	10
7	So sánh tổng hợp tất cả phương án	10
8	Phân tích pattern thất bại	11
8.1	Hai trạng thái policy học được	11
8.2	Nguyên nhân "Never HO"	11
8.3	Nguyên nhân "HO Badly"	11
8.4	Tại sao pre-trained model thoát được hai bẫy này?	11
9	Các vấn đề trong source code gốc	12
9.1	Bug nghiêm trọng: BS0 không bao giờ nhận reward	12
9.2	Bug nghiêm trọng: Model được save TRƯỚC khi train, và mặc định KHÔNG save	12
9.3	Bug cấu trúc: Default config phá vỡ 2-phase training	13
9.4	Bug nhỏ: n310 và n311 là class variable, không phải dataclass field	14
9.5	Vấn đề tiềm ẩn: l3_filter_w phụ thuộc l3_k nhưng không update khi l3_k thay đổi	14
9.6	Tóm tắt các vấn đề source code theo mức độ nghiêm trọng	15
10	Tại sao không thể tái tạo kết quả từ scratch — Phân tích toàn diện	15
10.1	Nguyên nhân nhóm 1 — Vấn đề thuật toán (Algorithm-Level)	15
10.1.1	Credit Assignment Problem — nguyên nhân gốc rễ	15
10.1.2	Thách thức dự đoán tương lai (9-step look-ahead)	16
10.2	Nguyên nhân nhóm 2 — Vấn đề trong source code gốc (Code-Level)	17
10.2.1	Script gốc không implement 2-phase training	17
10.2.2	Model không được lưu	17
10.2.3	Reward bias BS0	17
10.3	Nguyên nhân nhóm 3 — Thiếu thông tin từ paper (Documentation-Level)	17
10.3.1	Không có random seed	17
10.3.2	Không rõ số timesteps thực tế	17
10.3.3	WandB Sweep — khả năng dùng hyperparameter search	17
10.3.4	Không document dataset selection trong training	18
10.4	Sơ đồ nhân quả tổng hợp	18
10.5	Trả lời câu hỏi: "Tại sao pre-trained model hoạt động tốt?"	19
10.6	Điều kiện cần thiết để tái tạo được	19

1 Kết quả baseline (tái hiện thành công)

1.1 3GPP Baseline

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	99.756	41.83	3.37
50	99.751	50.00	2.05
70	99.797	47.26	2.53
90	99.696	44.41	6.99

Nhận xét: PP rate 42–50% — cứ 2 lần HO thì 1 lần ping-pong. 3GPP không thích nghi với tốc độ di chuyển của UE.

1.2 PPO gốc (pre-trained model của paper)

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	99.795	43.58	1.17
50	99.823	45.38	0.70
70	99.839	46.45	2.84
90	99.763	47.53	4.63

Sai số so với paper (Fig. 4): $< 0.02\%$ $\rightarrow$ Tái hiện evaluation thành công.

Policy của pre-trained model: Hoàn toàn deterministic (entropy $H = 0$):

```
best_BS=3 -> action=3, probs=[0., 0., 0., 1., 0.]
best_BS=1 -> action=1, probs=[0., 1., 0., 0., 0.]
best_BS=0 -> action=0, probs=[1., 0., 0., 0., 0.]
```

1.3 So sánh với paper

Method	Γ_R trung bình (%)	Match
Paper — 3GPP	~99.75	Đạt
Tái hiện — 3GPP	99.750	Đạt
Paper — PPO	~99.82	Đạt
Tái hiện — PPO gốc	99.805	Đạt

2 Vấn đề cốt lõi: Credit Assignment Problem

Đây là nguyên nhân gốc rễ khiến **tất cả 11 lần training thất bại**.

2.1 Tại sao policy bị stuck ở uniform distribution?

Với **uniform random policy** (mỗi BS được chọn với xác suất $1/N = 0.2$):

$$P(\text{HO hoàn thành trong 1 lần}) = P(5 \text{ bước liên tiếp cùng BS}) = \left(\frac{1}{5}\right)^4 = 0.16\%$$

Chuỗi hậu quả:

```
Uniform policy
|
v 0.16%/step HO trigger
P(HO) cực thấp -> UE luôn ở cùng pcell
|
v
Reward = sinr_norm[pcell] ~ 0.5 (không đổi)
|
v Không phụ thuộc action
Advantage A(s, a) ~ 0 cho mọi (s, a)
|
v
Policy gradient ~ 0 -> Policy không update
|
v
Stuck ở uniform distribution mãi mãi
```

Xác nhận thực nghiệm (300K steps, thử 3 giá trị ent_coef):

ent_coef	Entropy cuối	% of max (1.6094)
0.1	1.6094	100.0% — hoàn toàn uniform
0.01	1.6093	99.99%
0.001	1.6084	99.9%

→ Giảm entropy coefficient **không giúp được gì**. Vấn đề là gradient từ policy objective, không phải từ entropy regularization.

2.2 Ước tính tín hiệu gradient trong 5M steps

Metric	Tính toán	Giá trị ước tính
Số episodes (ep_len ≈ 291)	$5M/291$	~17,180 episodes
HOs/episode (uniform)	$291 \times 0.0016/9$	~0.05 HO/episode
Tổng HO completions	$17,180 \times 0.05$	~860 HOs
HOs / rollout buffer (2000)	$860/2500$	~0.34 HO/buffer

→ Chỉ ~860 HO completions trong **toàn bộ 5M steps** để học từ — tín hiệu quá thưa thớt.

2.3 Tại sao pre-trained model của paper lại hoạt động?

Model gốc có kết quả hoàn toàn deterministic ($H=0$), ngụ ý nó **đã hội tụ hoàn toàn**. Các giả thuyết:

1. **Random seed may mắn**: Initialization tình cờ tạo ra slight bias về một BS $\rightarrow$ positive feedback loop $\rightarrow$ hội tụ. Giả thuyết bị bác bỏ một phần khi thử 5 seeds đều thất bại.
2. **Train nhiều hơn 5M steps**: Paper không document rõ số steps thực tế.
3. **SINR reward ở cell edge**: Khi UE ở cell edge, SINR thấp $\rightarrow$ RLF penalty $-2C \rightarrow$ agent học cách tránh bằng cách commit cùng BS $\rightarrow$ HO xảy ra $\rightarrow$ gradient xuất hiện. Mechanism này có thể chỉ hoạt động với một số trajectory cụ thể.

2.4 Thách thức 9-timestep delay

Để HO thành công với $t_{ho_prep}=5 + t_{ho_exec}=4$, policy phải **dự đoán BS tốt nhất sau 9 bước tương lai**:

$$\hat{a}_t = \arg \max_i \text{SINR}_{i,t+9}$$

Nhưng observation tại t chỉ có SINR tại thời điểm t . MLP thuần túy không có khả năng look-ahead này, dẫn đến việc commit HO sai BS $\rightarrow$ target BS thay đổi trong khi HO đang thực hiện $\rightarrow$ RLF.

3 Thử nghiệm V1 — Training từ scratch

3.1 Thử nghiệm 1A: $t_{ho_prep}=3$ trong Phase 1

Ý tưởng: Giảm t_{ho_prep} từ 5 xuống 3 để tăng $P(\text{HO})$:

$$P(\text{HO} \mid t_{ho_prep} = 3) = (1/5)^2 = 4\% \quad (\text{tăng } 25\times \text{ so với } 0.16\%)$$

Config:

Phase 1: $t_{ho_prep}=3$, terminate_on_pp=False, terminate_on_rlf=True $\rightarrow$ 2.5M steps
Phase 2: $t_{ho_prep}=5$, terminate_on_pp=True, terminate_on_rlf=True $\rightarrow$ 2.5M steps

Kết quả training: entropy_loss = -1.61 (uniform) xuyên suốt

Kết quả eval:

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	~ 22	~ 0	~ 193
50	~ 10	~ 0	~ 186
70	~ 17	~ 5	~ 205
90	~ 29	~ 0	~ 196

Phân tích thất bại: Với $t_{ho_prep}=3$, HO xảy ra nhiều hơn nhưng đến BS **ngẫu nhiên** (không phải best BS) $\rightarrow$ SINR thấp $\rightarrow$ HOF $\rightarrow$ RLF. Reward quá noisy, gradient không đủ để thoát uniform.

3.2 Thử nghiệm 1B: 2-phase đúng paper (t_ho_prep=5 xuyên suốt)

Config:

```
Phase 1: t_ho_prep=5, terminate_on_pp=False, terminate_on_rlf=True -> 2.5M steps
Phase 2: t_ho_prep=5, terminate_on_pp=True, terminate_on_rlf=True -> 2.5M steps
lr_schedule = constant 1e-5, reset_num_timesteps=True
```

Kết quả training:

```
entropy_loss = -1.61 xuyên suốt TOÀN BỘ 5M steps (ca 2 phase)
Action probs: [0.199, 0.199, 0.202, 0.203, 0.197] <- UNIFORM
```

Root cause: ep_len_mean = 291 → RLF vẫn là nguyên nhân chính terminate episodes ngắn → gradient signal vẫn quá yếu. Phase 1 (no PP) không giúp vì RLF vẫn truncate.

4 Thử nghiệm V2 — 4 phương án giải quyết

Sau khi xác định root cause là credit assignment problem, 4 phương án được thử đồng thời.

4.1 Phương án Curriculum t_ho_prep = 1 → 3 → 5

Script: scripts/train_ppo_curriculum.py

Ý tưởng: Phase 0 với t_ho_prep=1 biến bài toán thành **multi-armed bandit** đơn giản:

$$P(\text{HO trigger/step} \mid t_{ho_prep} = 1) = 80\% \quad (\text{immediate reward})$$

Config:

Phase	t_ho_prep	t_ho_exec	terminate_rlf	terminate_pp	Steps
0	1 (10ms)	1 (10ms)	False	False	1M
1	3 (30ms)	2 (20ms)	True	False	2M
2	5 (50ms)	4 (40ms)	True	True	2M

Kết quả training Phase 0 — đột phá đầu tiên:

```
40K steps: entropy = -1.29 (so với -1.61 stuck trước đây!)
Phase 0: entropy: -1.61 -> -1.07
ep_rew: -1290 -> -232 -> +62 -> +335 -> +495 -> +3660
```

Đây là lần đầu tiên entropy thực sự giảm — policy đang học!

Kết quả training Phase 1: entropy: -1.07 → -1.58 (REGRESSION)

Kết quả training Phase 2: entropy: -1.58 → -1.61 (về lại UNIFORM)

Kết quả eval (t_ho_prep=5 — paper params):

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	29.5	97.1	68.2
50	12.6	95.7	64.5
70	23.7	95.9	63.6
90	25.3	95.7	68.9

Phân tích thất bại: Phase 0 thành công nhưng mỗi khi t_{ho_prep} tăng lên, credit assignment problem xuất hiện lại. Policy không thể ”giữ được” gì từ phase trước khi reward signal biến mất.

4.2 Phương án Reward Shaping

Script: scripts/train_ppo_reward_shaped.py

Ý tưởng: Thêm immediate reward dựa trên action chọn:

$$r_{shaped} = \alpha \cdot s_{SINR}[a_t], \quad \alpha = 0.1$$

Khi agent chọn best BS: $r_{shaped} \approx 0.1 \times 1.0 = 0.1$ Khi agent chọn worst BS: $r_{shaped} \approx 0.1 \times 0.0 = 0.0$

Kết quả eval:

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	36.5	25.0	40.0
50	22.7	41.3	173.9
70	23.8	34.7	124.5
90	17.8	26.3	122.8

Phân tích thất bại: $\alpha = 0.1$ quá nhỏ để break uniform distribution. Gradient từ SINR reward (không phụ thuộc action) vẫn dominant. Entropy vẫn stuck -1.61 .

4.3 Phương án Imitation Learning (BC + PPO)

Script: scripts/train_ppo_imitation.py

Thiết kế hai giai đoạn:

Stage 1 — Behavioral Cloning:

```
oracle_action = argmax(sinr_norm) # Luôn chọn BS có SINR cao nhất
dataset: 348,400 samples từ 57 datasets training
Training: 20 epochs, batch_size=512, lr=3e-4
```

Kết quả BC:

```
Epoch 1: loss=0.2061, acc=98.9%
Epoch 2: loss=0.0004, acc=100.0%
Epoch 3+: loss~0.0000, acc=100.0% (hoàn toàn deterministic sau 2 epochs!)
```

Stage 2 — PPO Fine-tuning:

```
ent_coef = 0.01 (giảm để preserve BC structure)
Phase 1: 2.5M steps, terminate_on_pp=False
Phase 2: 2.5M steps, terminate_on_pp=True
```

Kết quả eval:

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	38.8	0.0	0.0
50	41.3	0.0	0.0
70	50.0	0.0	0.0
90	44.3	0.0	0.0

Phân tích: PP=0%, RLF=0% → tốt về phụ, nhưng $\Gamma_R = 38-50\%$ → **"Never HO" policy**. UE kẹt ở BS ban đầu không theo kịp channel thay đổi. Nguyên nhân: PPO Phase 2 với terminate_on_pp=True → mọi HO đều có rủi ro PP penalty → agent học "tránh HO hoàn toàn".

Điểm tích cực: BC hoạt động hoàn hảo, ent_coef=0.01 preserve structure deterministic.

4.4 Phương án Multiple Seeds

Script: scripts/train_ppo_multiseed.py

Ý tưởng: Paper có thể đã dùng lucky seed.

Seeds thử: [0, 42, 123, 777, 1234] $\times 2M$ steps

Kết quả:

```
Seed 0: entropy = 1.6093 (uniform)
Seed 42: entropy = 1.6094 (uniform)
Seed 123: entropy = 1.6094 (uniform)
Seed 777: entropy = 1.6093 (uniform)
Seed 1234: entropy = 1.6090 (uniform, "best" -- vẫn uniform)
```

Kết luận: Credit assignment problem là **systematic**, không phải do seed. Với 2M steps standard 2-phase training, tất cả seeds đều stuck uniform.

4.5 Tổng hợp V2

Model	Γ_R TB (%)	PP TB (%)	RLF TB (%)	Trạng thái
Paper PPO (original)	99.8	45.7	2.3	TARGET
3GPP baseline	99.75	45.9	3.7	Reference
PPO 2-phase (paper)	19.6	1.1	195.0	Uniform
Curriculum 1 → 3 → 5	22.8	96.1	66.3	PP=97%
Reward shaping $\alpha = 0.1$	25.2	31.8	115.0	Uniform
Imitation (BC+PPO)	43.6	0.0	0.0	Never HO
Multiseed (5 seeds)	20.7	49.4	93.2	Uniform

5 Thử nghiệm V3 — BC + Gradual Curriculum

Script: scripts/train_ppo_bc_curriculum.py

Thiết kế: Kết hợp ưu điểm của BC (initialization tốt) và Curriculum (transition mượt):

```
BC init (oracle: argmax sinr_norm, 30 epochs) -> policy deterministic
-> Phase 0: prep=1, exec=1, no-RLF, no-PP -> 1M steps (warm up)
-> Phase 1: prep=2, exec=1, RLF, no-PP -> 1M steps
-> Phase 2: prep=3, exec=2, RLF, no-PP -> 1M steps
-> Phase 3: prep=4, exec=3, RLF, PP -> 1M steps (PP introduced)
-> Phase 4: prep=5, exec=4, RLF, PP -> 1M steps (paper params)
Total: 5M PPO steps
```

Key design choices:

- $\text{ent_coef}=0.01$ — preserve BC structure
- $t_{\text{ho_exec}}$ giảm dần từ 1 $\rightarrow$ 4: ít disconnection time ở đầu $\rightarrow$ ít HOF
- PP chỉ từ Phase 3: học HO trước, học tránh PP sau
- lr giảm dần: $3e-5 \rightarrow 2e-5 \rightarrow 1.5e-5 \rightarrow 1e-5 \rightarrow 5e-6$

Root cause phát hiện trong V3: $\text{ent_coef}=0.01$ quá thấp $\rightarrow$ entropy collapse nhanh $\rightarrow$ policy stuck ”never HO”. Paper dùng $\text{ent_coef}=0.1$ (cao hơn 10 lần).

Kết quả eval BC+Curriculum Phase 0 ($t_{\text{ho_prep}}=1$, $\text{ent_coef}=0.01$):

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	38.8	0.0	0.0
50	41.3	0.0	0.0
70	50.0	0.0	0.0
90	44.3	0.0	0.0

Kết quả eval BC+Curriculum Final ($t_{\text{ho_prep}}=5$, paper params):

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	31.3	0.0	225.0
50	37.4	0.0	294.7
70	25.8	0.0	235.0
90	21.1	0.0	239.1

Nhận xét: Phase 0 $\rightarrow$ ”Never HO”. Final $\rightarrow$ ”HO badly” (RLF=225–295%). Entropy collapse quá sớm với $\text{ent_coef}=0.01$ là nguyên nhân.

6 Thử nghiệm V4 — Biến thể BC+Curriculum

Cải tiến chính: $\text{ent_coef}=0.1$ (giá trị paper), SubprocVecEnv $n_{\text{envs}}=4$, GPU

6.1 bc_paper — BC + paper params trực tiếp

Script: scripts/train_ppo_bc_paper.py **Hypothesis:** BC giải quyết cold-start problem, paper params đủ tốt từ đó.

Kết quả eval:

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	24.1	0.0	250.0
50	33.2	0.0	490.0
70	31.1	0.0	325.0
90	30.6	0.0	361.5

Pattern: "HO badly" — cố HO nhưng thất bại liên tục. Target BS thay đổi trong 9 timesteps $\rightarrow$ RLF.

6.2 bc_highent — BC + Curriculum + ent_coef=0.1

Script: scripts/train_ppo_bc_highent.py **Khác V3:** ent_coef=0.1 thay vì 0.01

Kết quả eval:

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	9.1	0.0	225.0
50	21.8	14.3	285.7
70	26.6	23.8	228.6
90	27.9	5.3	278.9

Nhận xét: Γ_R thấp nhất trong V4. Entropy cao hơn $\rightarrow$ explore nhiều HO hơn $\rightarrow$ nhiều RLF hơn. Có PP=14–24% ở 50–70 km/h.

6.3 bc_noterm — BC + Curriculum + không terminate

Script: scripts/train_ppo_bc_noterm.py **Khác biệt:** terminate_on_rlf=False, terminate_on_pp=False xuyên suốt (chỉ dùng reward penalty)

Rationale: Không terminate $\rightarrow$ policy học từ hậu quả thay vì bị reset

Kết quả eval:

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	38.8	0.0	0.0
50	41.3	0.0	0.0
70	50.0	0.0	0.0
90	44.3	0.0	0.0

Pattern: Giống hệt Imitation V2 — "Never HO". Policy học "không HO còn an toàn hơn" dù không bị terminate vì HO cost (disconnection) vẫn âm.

6.4 bc_2m — BC + Curriculum + 2M steps/phase

Script: scripts/train_ppo_bc_2m.py **Khác biệt:** 2M steps/phase (10M tổng), ent_coef=0.1

Kết quả eval:

Speed (km/h)	Γ_R (%)	PP rate (%)	RLF rate (%)
30	22.9	0.0	250.0
50	35.1	0.0	490.0
70	31.1	0.0	350.0
90	30.6	0.0	361.5

Nhận xét: Pattern giống hệt bc_paper (cùng RLF=250–490%). Thêm steps không giúp được — cùng fundamental problem.

6.5 Tổng hợp V4

Model	Γ_R TB (%)	PP TB (%)	RLF TB (%)	Pattern
bc_paper	29.8	0	357	HO badly
bc_highent	21.4	10.9	254	HO badly + PP
bc_noterm	43.5	0	0	Never HO
bc_2m	29.9	0	363	HO badly

7 So sánh tổng hợp tất cả phương án

#	Method	Γ_R TB (%)	PP TB (%)	RLF TB (%)	Trạng thái
0	Paper PPO (original)	99.81	45.7	2.3	TARGET
0	3GPP baseline	99.75	45.9	3.7	Reference
1	PPO 2-phase (paper params)	19.6	1.1	195.0	Uniform
2	PPO t_ho_prep=3 Phase 1	~22	~0	~195	Uniform
3	Curriculum 1 → 3 → 5	22.8	96.1	66.3	PP=97%
4	Reward shaping $\alpha = 0.1$	25.2	31.8	115.0	Uniform
5	Imitation (BC+PPO)	43.6	0.0	0.0	Never HO
6	Multiseed (5 seeds × 2M)	20.7	49.4	93.2	Uniform
7	BC+Curriculum ent=0.01	28.9	0.0	248.5	HO badly
8	bc_paper (BC+paper params)	29.8	0.0	356.6	HO badly
9	bc_highent (BC+ent=0.1)	21.4	10.9	254.6	HO badly
10	bc_noterm (BC+no-term)	43.5	0.0	0.0	Never HO
11	bc_2m (BC+2M steps)	29.9	0.0	362.9	HO badly

Kết quả cao nhất của các phương án từ scratch: $\Gamma_R = 43.6\%$ — so với target **99.81%**

8 Phân tích pattern thất bại

8.1 Hai trạng thái policy học được

Sau 11 lần thử, chỉ có **hai trạng thái ổn định** mà policy hội tụ vào:

POLICY STATE SPACE	
+-----+ "Never HO" +-----+ G_R = 38-50% PP = 0% RLF = 0% Xuất hiện khi: - terminate PP - no-term + BC +-----+	+-----+ "HO Badly" +-----+ G_R = 22-35% PP = 0-11% RLF = 250-490% Xuất hiện khi: - ent_coef cao - t_ho_prep lon - reward HO > cost +-----+
TARGET: G_R = 99.8% (chi pre-trained model dat duoc)	

8.2 Nguyên nhân "Never HO"

- Khi `terminate_on_pp=True`: mọi HO đều có rủi ro PP penalty → agent học "tránh HO hoàn toàn" để tránh rủi ro
- Khi BC init + penalty-only (no terminate): HO có cost tức thì (disconnection) nhưng reward dài hạn khó học → agent an toàn ở "Never HO"

8.3 Nguyên nhân "HO Badly"

Với 9-timestep delay (5 prep + 4 exec):

```
t=0: Agent commit HO sang BS_i vì BS_i có SINR cao nhất
t=9: HO execution xong, UE kết nối BS_i
      Nhưng UE đã di chuyển! BS_i không còn là best BS
      SINR[BS_i, t=9] < Q_in -> HOF -> RLF
```

Tần suất RLF=490% có nghĩa là UE cứ mỗi 1 HO thành công thì có thêm 4.9 lần RLF. Điều này cực kỳ tệ.

8.4 Tại sao pre-trained model thoát được hai bẫy này?

Phân tích behavior của pre-trained model:

- **Deterministic** ($H=0$): policy không bao giờ "không chắc" — luôn chọn cùng một BS với xác suất 1.0
- Điều này ngụ ý model có thể **dự đoán được BS tốt nhất trong tương lai** từ pattern SINR
- Khả năng: model đã học được **implicit look-ahead** từ gradient signal — nhưng signal này chỉ xuất hiện được nếu HO xảy ra đủ nhiều để tạo ra learning signal ban đầu

9 Các vấn đề trong source code gốc

Phần này phân tích trực tiếp từ source code, không dựa vào ghi chú. Mỗi vấn đề được dẫn đến file và dòng cụ thể.

9.1 Bug nghiêm trọng: BS0 không bao giờ nhận reward

File: src/ho_optim_drl/gym_env/ho_env_ppo.py, dòng 359

```
# Code gốc -- SAI
def _get_reward(self) -> float:
    ...
    if self.s_pcell[-1] > 0:          # <- điều kiện sai
        reward += sinr_norm[self.s_pcell[-1]].item()
    if self.s_pcell[-1] == best_bs:
        reward += self.config.rew_const
```

Phân tích chi tiết:

`self.s_pcell[-1]` là index của BS đang serve (pcell). BS0 có index = 0.

Điều kiện $0 > 0 \rightarrow \text{False} \rightarrow$ **BS0 không bao giờ nhận SINR reward.**

Cụ thể, khi pcell = BS0:

- Không nhận `sinr_norm[0]` (SINR reward)
- Không nhận $+C$ bonus dù BS0 là best BS
- Reward = 0 thay vì `sinr_norm[0] + 0.95`

Điều kiện đúng phải là: `if self.s_pcell[-1] >= 0:`

Ảnh hưởng thực tế trong training:

Tương tự như nghiêm trọng nhưng tác động bị giảm thiểu bởi một cơ chế ngẫu nhiên: mỗi episode, pcell ban đầu được chọn là `np.argmax(rsrp)` tại bước đầu của dataset. Vì training dùng nhiều datasets (30 km/h và 50 km/h), vị trí UE khởi đầu khác nhau nên tần suất BS0 là pcell không quá cao.

Tuy nhiên, trong evaluation, bias này vẫn tồn tại có hệ thống: mọi timestep UE kết nối BS0 đều không có reward $\rightarrow$ agent bị khuyến khích **rời BS0 sớm hơn cần thiết**, kể cả khi BS0 đang có SINR tốt.

9.2 Bug nghiêm trọng: Model được save TRƯỚC khi train, và mặc định KHÔNG save

File: scripts/train_ppo.py, dòng 20 và 141–144

```
# Dòng 20
SAVE_MODEL = False          # <- mac dinh: KHONG save gi ca

# Dòng 141-144
if SAVE_MODEL:
    model.save(model_dir)    # <- save DAY (truoc khi train!)

model.learn(total_timesteps=config.n_steps_total, progress_bar=True) # <- train DAY
```

Đây là hai lỗi độc lập trong cùng một đoạn:

Lỗi 1 — SAVE_MODEL = False: Ai chạy python run.py train_ppo sẽ train 5M steps xong và không có file model nào được lưu. Kết quả training biến mất hoàn toàn.

Lỗi 2 — Save trước learn(): Kể cả khi đổi thành SAVE_MODEL = True, lệnh model.save() ở dòng 142 được gọi **trước** model.learn() ở dòng 144. Model được save tại thời điểm này là model chưa qua bất kỳ bước training nào — chỉ có random weight initialization.

Code đúng phải là:

```
SAVE_MODEL = True          # bat luu model

model.learn(total_timesteps=config.n_steps_total, progress_bar=True)

if SAVE_MODEL:
    model.save(model_dir)    # save SAU KHI train xong
```

Hệ quả với việc tái tạo: Bất kỳ ai cố tái tạo paper bằng cách chạy script gốc đều sẽ không lưu được model. Đây là lý do quan trọng nhất khiến việc reproduce trở nên khó khăn về mặt thực tế.

9.3 Bug cấu trúc: Default config phá vỡ 2-phase training

File: src/ho_optim_drl/config.py, dòng 60

```
@dataclass
class Config:
    ...
    terminate_on_pp: bool = True    # <- default = True (Phase 2 config!)
    terminate_on_rlf: bool = True
```

Vấn đề:

Paper mô tả 2-phase training:

- Phase 1: terminate_on_pp = False (agent học HO tự do, không bị terminate vì PP)
- Phase 2: terminate_on_pp = True (agent học tránh PP)

Nhưng default config có terminate_on_pp = True — là Phase 2 config.

Khi chạy script gốc train_ppo.py, không có chỗ nào thay đổi terminate_on_pp. Toàn bộ 5M steps đều chạy với Phase 2 config:

```
def train_ppo(root_path: str):
    config = Config()          # terminate_on_pp=True ngay tu dau
    ...
    env = HandoverEnvPPO(config, ...)
    model.learn(total_timesteps=config.n_steps_total, ...)
    # Không có Phase 1. Không có Phase 2. Chỉ một phase duy nhất với Phase 2 config.
```

Ý nghĩa: Script gốc không implement 2-phase training như paper mô tả. Paper mô tả 2-phase nhưng code gốc chỉ có 1-phase với Phase 2 parameter. Đây là **sự không nhất quán giữa paper và code**.

9.4 Bug nhỏ: n310 và n311 là class variable, không phải dataclass field

File: src/ho_optim_drl/config.py, dòng 37–38

```
@dataclass
class Config:
    ...
    t_t310: int = 1_000 // delta_t_ms # dataclass field (có type annotation)
    n310 = 10                          # class variable (không có type annotation)
    n311 = 3                          # class variable (không có type annotation)
```

Trong Python, @dataclass chỉ nhận các biến **có type annotation** làm instance fields. Biến không có type annotation trở thành **class variable** — tồn tại ở cấp class, không phải ở cấp instance.

So sánh:

```
config1 = Config()
config2 = Config()

config1.t_t310 = 200 # thay đổi chỉ instance config1 -> OK
config2.t_t310      # vẫn = 100 -> OK

config1.n310 = 5     # tạo instance attribute ở config1 -> OK nhưng không nhất quán
Config.n310          # vẫn = 10 (class variable không đổi)
config2.n310         # = 10 (đọc từ class variable)
```

Hậu quả thực tế:

- config.update({'n310': X}) hoạt động được (vì hasattr tìm thấy qua class) nhưng tạo ra instance attribute shadow class variable — không nhất quán
- Hai instance Config có thể có n310 khác nhau theo cách không rõ ràng
- Khi copy config hoặc serialize, n310 và n311 có thể bị bỏ sót vì không nằm trong __dataclass__-fields__

9.5 Vấn đề tiềm ẩn: l3_filter_w phụ thuộc l3_k nhưng không update khi l3_k thay đổi

File: src/ho_optim_drl/config.py, dòng 22–23

```
l3_k: int = 16
l3_filter_w: float = 1 / (2 ** (l3_k / 4)) # = 1/16 = 0.0625
```

Trong Python dataclass, default value của một field được tính **một lần duy nhất tại class definition time**, không phải tại instance creation time. Biểu thức $1 / (2 ** (l3_k / 4))$ được tính với $l3_k = 16 \rightarrow l3_filter_w = 0.0625$ cố định.

```
config = Config()
config.l3_k = 8 # đổi l3_k
config.l3_filter_w # vẫn = 0.0625, không tu update!
```

Tác động trong thực nghiệm này: Không ảnh hưởng vì chúng ta không thay đổi l3_k. Nhưng đây là hidden coupling cần biết.

9.6 Tóm tắt các vấn đề source code theo mức độ nghiêm trọng

#	Vấn đề	File : Dòng	Mức độ	Ảnh hưởng thực tế
1	BS0 không nhận reward (> 0 thay vì ≥ 0)	ho_env_ - ppo.py:359	Cao	Bias training, khuyến khích rời BS0 sớm
2	SAVE_MODEL=False — không lưu model	train_ - ppo.py:20	Cao	Mất kết quả training hoàn toàn
3	Model save TRƯỚC model.learn()	train_ - ppo.py:141--144	Cao	File lưu là random model chưa train
4	Default terminate_on_pp=True — không có Phase 1	config.py:60	Vừa	Script gốc không làm 2-phase như paper
5	n310, n311 là class variable	config.py:37--38	Thấp	Inconsistency, không gây crash
6	Training chỉ dùng dataset 0	ho_env_ppo.py + train_ - ppo.py	Vừa	Thiếu diversity, overfitting tiềm ẩn
7	l3_filter_w không update theo l3_k	config.py:22--23	Thấp	Không ảnh hưởng nếu không đổi l3_k

10 Tại sao không thể tái tạo kết quả từ scratch — Phân tích toàn diện

Đây là phân tích có cấu trúc về **tại sao** tất cả 11 lần thử đều thất bại. Có ba nhóm nguyên nhân độc lập, mỗi nhóm một mình đã đủ để gây thất bại.

10.1 Nguyên nhân nhóm 1 — Vấn đề thuật toán (Algorithm-Level)

10.1.1 Credit Assignment Problem — nguyên nhân gốc rễ

Đây là vấn đề lý thuyết cơ bản nhất. Với thiết kế hiện tại:

Bước 1: Random policy → HO không xảy ra

$$\begin{aligned}
 P(\text{HO trigger}) &= P(\text{chọn cùng 1 BS trong } t_{ho_prep} \text{ bước liên tiếp}) \\
 &= \left(\frac{1}{N}\right)^{t_{ho_prep}-1} = \left(\frac{1}{5}\right)^4 = 0.0016 = 0.16\%
 \end{aligned}$$

Trong 2000 bước của 1 rollout buffer: kỳ vọng chỉ $2000 \times 0.0016 = 3.2$ HO triggers. Nhưng mỗi HO mất thêm 9 bước để hoàn thành (5 prep + 4 exec), và với uniform policy, target BS thay đổi trong lúc đó → HO fail → RLF → episode kết thúc sớm.

Bước 2: HO không xảy ra → Reward không phụ thuộc action

Khi pcell không thay đổi (không HO), reward tại mỗi bước chỉ phụ thuộc vào SINR của pcell hiện tại:

$$r_t = s_{SINR}[\text{pcell}] \approx 0.5 \quad (\text{không đổi})$$

Action a_t không ảnh hưởng đến pcell → không ảnh hưởng đến reward.

Bước 3: Reward không phụ thuộc action $\rightarrow$ Advantage ≈ 0

Advantage function:

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t) = \mathbb{E}[r_t + \gamma r_{t+1} + \dots \mid s_t, a_t] - V(s_t)$$

Khi r không phụ thuộc a : $Q(s, a) = V(s) \rightarrow A(s, a) \approx 0$ với mọi (s, a) .

Bước 4: Advantage $\approx 0 \rightarrow$ Policy gradient $\approx 0 \rightarrow$ Policy không update

PPO policy loss:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

Khi $\hat{A}_t \approx 0$: $L^{CLIP} \approx 0 \rightarrow \nabla_{\theta} L^{CLIP} \approx 0 \rightarrow$ policy weight không thay đổi.

Kết quả: Policy bị kẹt tại điểm khởi tạo (gần uniform) vĩnh viễn, không phụ thuộc số steps.

Xác nhận thực nghiệm (300K steps):

ent_coef	Entropy	% of max uniform	Verdict
0.1	1.6094	100.0%	Hoàn toàn uniform
0.01	1.6093	99.99%	Gần như uniform
0.001	1.6084	99.9%	Vẫn uniform

Entropy coefficient không giúp được vì vấn đề nằm ở policy loss gradient, không phải entropy regularization.

10.1.2 Thách thức dự đoán tương lai (9-step look-ahead)

Ngay cả khi agent vượt qua được credit assignment problem và bắt đầu học HO, nó vẫn đối mặt với một thách thức cấu trúc khác.

Khi agent commit HO tại bước t , kết quả (UE kết nối BS mới) chỉ biết tại bước $t + 9$:

$$t_{HO_complete} = t_{commit} + t_{ho_prep} + t_{ho_exec} = t + 5 + 4 = t + 9$$

Trong 90 ms đó, kênh truyền thay đổi. Để HO thành công, agent cần chọn BS sao cho **tại thời điểm** $t + 9$, BS đó vẫn có SINR tốt:

$$a_t^* = \arg \max_i \text{SINR}_i(t + 9)$$

Nhưng observation tại t chỉ có:

$$s_t = [s_{BS}, s_{\text{SINR}}(t), s_{PP}]$$

Không có thông tin về $\text{SINR}(t + 1), \dots, \text{SINR}(t + 9)$. MLP không có memory hay look-ahead — nó chỉ ánh xạ $s_t \rightarrow a_t$ dựa trên t hiện tại.

Hệ quả quan sát được (V4 experiments):

- bc_paper: RLF = 250–490% $\rightarrow$ agent commit HO nhưng target BS thay đổi sau 9 bước $\rightarrow$ HOF $\rightarrow$ RLF liên tục
- Không có variant nào thoát khỏi pattern này trừ khi "never HO"

10.2 Nguyên nhân nhóm 2 — Vấn đề trong source code gốc (Code-Level)

10.2.1 Script gốc không implement 2-phase training

Như đã phân tích ở §9.3:

- Paper mô tả Phase 1 với `terminate_on_pp=False`
- Default config có `terminate_on_pp=True`
- `train_ppo.py` không thay đổi config → toàn bộ training là Phase 2

Điều này có nghĩa: **bất kỳ ai chạy `python run.py train_ppo` đều không đang tái tạo paper** dù họ không biết điều này.

10.2.2 Model không được lưu

Ngay cả khi training hội tụ, `SAVE_MODEL=False` đảm bảo không có file model nào được tạo. Script gốc **cố tình** không lưu model (có thể vì paper dùng WandB sweep để lưu).

10.2.3 Reward bias BS0

Bias reward đối với BS0 tạo ra một "lực kéo" không nhất quán: agent có thể học policy phụ thuộc vào việc có hay không có BS0 trong episode, thay vì học policy thuần túy dựa trên SINR.

10.3 Nguyên nhân nhóm 3 — Thiếu thông tin từ paper (Documentation-Level)

Đây là các thông tin **không được document** trong paper hoặc repo, khiến việc tái tạo về nguyên tắc là không đầy đủ.

10.3.1 Không có random seed

Paper không document seed nào được dùng để train pre-trained model. Kết quả thực nghiệm multi-seed ($5 \text{ seeds} \times 2\text{M steps}$, §4.4) chứng minh rằng vấn đề không phải seed-sensitive với 5 seeds đã thử. Nhưng không thể thử tất cả 2^{32} seeds.

10.3.2 Không rõ số timesteps thực tế

Paper ghi `n_steps_total = 5,000,000` trong config. Nhưng:

- Không biết paper có train nhiều lần và chọn model tốt nhất không
- Không biết có early stopping không
- Không biết có fine-tuning sau không

10.3.3 WandB Sweep — khả năng dùng hyperparameter search

Trong `train_ppo.py`:

```
def get_sweep_config():
    return {
        "name": SWEEP_NAME,
        "method": "bayes",          # Bayesian optimization sweep!
        "metric": {"goal": "maximize", "name": "reward_sum_avg"},
```

```

    "parameters": {
        "ent_coef": {"values": [0.001, 0.01, 0.1]},
        "rew_const": {"values": [0.8, 0.9, 1.0]},
    },
}

```

Code đã setup sẵn **WandB Bayesian sweep** cho `ent_coef` và `rew_const`. Điều này gợi ý rằng paper model có thể được train qua **hyperparameter search**, không phải single run với default params.

Nếu paper chạy 9 combinations ($3 \text{ ent_coef} \times 3 \text{ rew_const}$) và chọn model tốt nhất, xác suất ít nhất 1 run hội tụ tăng lên đáng kể. Đây là **undocumented training procedure** quan trọng nhất.

10.3.4 Không document dataset selection trong training

Như đã phân tích ở §9.6, script gốc chỉ dùng dataset 0. Nhưng paper mô tả "training speeds: 30, 50 km/h" ngụ ý dùng cả hai. Không rõ paper thực sự train trên bao nhiêu datasets và theo thứ tự nào.

10.4 Sơ đồ nhân quả tổng hợp

```

    TAI SAO KHONG TAI TAO DUOC -- TOAN CANH

    NHOM 1: THUAT TOAN (khong the tranh voi thiet ke hien tai)
    +-----+
    | Random init -> H0 xac suat 0.16% |
    | -> Reward khong phu thuoc action |
    | -> Advantage ~ 0 -> Gradient ~ 0 |
    | -> Policy stuck tai uniform mai mai |
    | |
    | Neu vuot qua duoc: 9-step delay |
    | -> Can du doan SINR(t+9) tu SINR(t) |
    | -> MLP khong co memory -> HOF -> RLF=250-490% |
    +-----+

    NHOM 2: CODE (gay ra boi bugs trong source goc)
    +-----+
    | SAVE_MODEL=False -> khong luu model du train thanh cong |
    | Save truoc learn() -> luu model chua train |
    | Default terminate_on_pp=True -> khong co Phase 1 |
    | BS0 reward bug -> bias training |
    +-----+

    NHOM 3: DOCUMENTATION (thieu thong tin tu paper)
    +-----+
    | Khong co seed -> khong the reproduce exact run |
    | WandB sweep khong duoc document -> paper co the |
    | da chay 9+ combinations va chon best |
    | Dataset selection khong ro |
    | So timesteps thuc te khong ro |
    +-----+

    KET LUAN: Ca 3 nhom cung tac dong -> reproduction khong
    kha thi voi thong tin hien co, ngay ca ve ly thuyet.

```

10.5 Trả lời câu hỏi: "Tại sao pre-trained model hoạt động tốt?"

Pre-trained model đạt $\Gamma_R = 99.8\%$ với policy hoàn toàn deterministic ($H=0$). Đây là trạng thái hội tụ hoàn toàn. Dựa trên phân tích code, có một số giả thuyết được sắp xếp theo xác suất:

Giả thuyết 1 (Xác suất cao): WandB sweep chọn lucky combination

Code đã setup sẵn Bayesian sweep cho 9 combinations. Nếu paper chạy sweep và chọn model tốt nhất:

- Mỗi combination có xác suất nhỏ hội tụ
- 9 combinations cho xác suất tổng hợp cao hơn đáng kể
- Đây là cách thông thường để làm paper trong ML

Giả thuyết 2 (Xác suất trung bình): Số steps nhiều hơn 5M

5M steps chỉ tạo ra ~ 860 HO completions. Nếu paper train 50M steps, số HOs tăng lên ~ 8600 , có thể đủ để gradient signal tích lũy. Config có `n_steps_total = 5e6` nhưng đây là default, không phải giá trị thực tế của paper run.

Giả thuyết 3 (Xác suất thấp): Lucky seed với specific trajectory

Một số trajectories trong dataset có "cell edge" scenarios rõ ràng hơn — UE đi qua vùng biên BS và SINR drop mạnh. Ở những đoạn đó, RLF penalty buộc agent học cách commit HO. Với seed may mắn tạo ra initialization bias về một BS, positive feedback loop có thể bắt đầu.

Giả thuyết 4 (Xác suất rất thấp): Paper chỉ train 1 lần và may mắn

Không thể loại trừ khả năng paper chạy 1 lần với default seed=0, ngẫu nhiên hội tụ, và không kiểm tra reproducibility. Đây là vấn đề phổ biến trong ML papers.

10.6 Điều kiện cần thiết để tái tạo được

Để tái tạo $\Gamma_R \geq 99\%$ từ scratch, cần giải quyết tất cả ba nhóm vấn đề:

Nhóm 1 (Thuật toán) — cần ít nhất 1 trong:

- ☐ Thêm SINR look-ahead vào observation: s_t^{new} bao gồm $\text{SINR}(t+1), \dots, \text{SINR}(t+9)$
- ☐ Reward shaping mạnh: $r_{shaped} = \alpha \cdot \text{SINR}_{action}(t+9)$
- ☐ Curriculum với `t_ho_prep`: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, đủ steps mỗi phase để knowledge transfer
- ☐ Dùng LSTM/GRU thay MLP để agent học temporal pattern tự nhiên

Nhóm 2 (Code) — cần fix tất cả:

- Fix `> 0` thành `>= 0` ở reward function (đã biết)
- Đặt `model.save()` sau `model.learn()` (đã biết)
- Implement Phase 1 với `terminate_on_pp=False` (đã làm trong các scripts tùy chỉnh)

Nhóm 3 (Documentation) — không thể fix hoàn toàn:

- ☐ Cần author của paper cung cấp seed, số steps thực tế, và WandB sweep history
- ☐ Hoặc cần brute-force thử nhiều seeds/hyperparameters hơn

*Báo cáo này tổng hợp kết quả từ 11 lần thử nghiệm trong khoảng thời gian 2026-06-18 đến 2026-07-01.
Kết luận chính: Pre-trained model của paper có thể evaluate được (khớp $< 0.02\%$), nhưng không thể tái hiện từ scratch trong 5M–10M steps do Credit Assignment Problem cố hữu của bài toán HO với $t_{ho_prep}=5$.*